

Assignment 3: Style Transfer of Images

By: Julia Reinhart

- 1. Introduction:** Style Transfer involves combining the content of one image with the style of another. Content can be defined as the existing structure and items of the image while style can be defined as the strokes used in the image along with the color scales. In the cited paper by Gatys et al., convolutional neural networks were used to separate the content of one image with the style of another by utilizing deep features. This paper examines a similar experiment where we analyze the different hyperparameters such as style weight, number of steps, and content layer, and determine their effect in the resulting image.
- 2. Implementation:** I used the pretrained version of the VGG19 model without adjusting any weights. This is because I was not using the model to classify images and so the classifier layers were not needed. Instead, image representations were extracted so that the target image could be classified. The code to accomplish this is shown below:

```
# get the "features" portion of VGG19 (we will not need the "classifier" portion)
```

```
vgg = models.vgg19(pretrained=True).features
```

```
# freeze all VGG parameters since we're only optimizing the target image
```

```
for param in vgg.parameters():
```

```
    param.requires_grad_(False)
```

If cuda was available, then GPU was used. Otherwise, CPU was used. In my case, CPU was used which resulted in a longer time to run the model and extract the necessary features.

For the content image, a picture of a static blue house was used (saved as houseContent.jpg in the GitHub Repo). For the style image, a Van Gogh painting was used (saved as vanGoghStyle.png in the GitHub Repo). The style image enhanced the image with artistic textures, a color scheme, and brush strokes. Continued on the next page is the content image along with the style image.

Content Image:



Style Image:



Both images were resized and normalized to minimize the impact of different sizes of each image distorting the outcome of the model.

For content and style feature extraction, different layers were designed to extract different features. Similar to a CNN used for image processing, later layers have more capability to extract advanced features while earlier layers extract single, simplistic features. For example, the earlier layers conv1_1 and conv2_1 extract edges, textures, and fine details. The middle layers, conv3_1 and conv4_1, represent shapes and patterns. Finally, the later layers such as conv4_2 and conv5_1 are able to learn higher features such as objects and structure. The code to assign these layers is shown below.

if layers is None:

```
layers = {  
    '0': 'conv1_1',  
    '5': 'conv2_1',  
    '10': 'conv3_1',  
    '19': 'conv4_1',  
    '21': 'conv4_2',  
    '28': 'conv5_1'  
}
```

These layers served as a dictionary and were mapped to index values using the same names as the Gatys paper. One benefit to using these layers was that we could compare shallow, middle, and deeper VCG19 representations from the images.

To obtain the content representation, feature activations were used from the content image and target image. The initial notebook began with a default value of conv4_2 for content which was the exact value used in the sample paper. Using the content layer as conv4_2 preserves the structure of the image more than the pixel values. For content, there was a loss which examined the resultant image features, called the target image, to the original content image features. The mean squared error measured this loss. The code to calculate the content loss is shown below.

```
# calculate the content loss
```

```
content_loss = torch.mean((target_features['conv5_1'] - content_features['conv5_1'])**2)
```

Different content layers were also tested which portrayed how shallow, medium, and deep layers of the neural network differentiate the final target image. We explain this in more detail in the experiments section of the paper.

To determine the style representation, multiple VCC19 layers were used as opposed to a single layer that was used in the content representation. The code which determined the style weights is illustrated bellow.

```
style_weights = {'conv1_1': 1.,  
                 'conv2_1': 0.8,  
                 'conv3_1': 0.5,  
                 'conv4_1': 0.3,  
                 'conv5_1': 0.1}
```

Style is extracted in each of these differing layers. As stated earlier, earlier layers learn low level textures, colors, and edges while deeper layers have the capacity to understand advanced recognition such as shapes and objects. Layer conv4_2 is left out from the style layers because it is contained in the default content representation.

The style features are calculated a single time using the following code:

```
style_features = get_features(style, vgg)
```

Next, these features get transformed into Gram matrices for each layer of our style representation with this code below:

```
style_grams = {layer: gram_matrix(style_features[layer]) for layer in style_features}
```

The gram_matrix function captured the image style by obtaining the relationship between the feature maps within a single convolutional layer. This allows the model to understand textures and patterns apart from the pixel's spatial location, making it an effective component of separating style from the content of the image.

First, the tensor dimensions are extracted using this line of code:

```
batch_size, d, h, w = tensor.size()
```

Then each the height and width dimensions are flattened into a single dimension using this code:

```
tensor = tensor.view(d, h * w)
```

Finally, the flattened version of the matrix is multiplied by the transpose version of that matrix using this code:

```
gram = torch.mm(tensor, tensor.t())
```

The final matrix outcome of this is a matrix with dimensions d x d where both rows and columns represent features and the values represent how strongly these features are correlated with each other. In effect, the gram matrix shows which visual patterns appear together in the image

regardless of the location of those patterns since we are extracting style using the gram matrix not content.

Then, a total loss function was used that took into account content loss and style loss. The content loss determined the location of what was in the target image compared with the content image while the style loss measured the distance between the target image's gram matrices to the style image's Gram matrices.

For every single style layer within the style weights, these lines of code executed:

```
target_gram = gram_matrix(target_feature)
style_gram = style_grams[layer]
layer_style_loss = style_weights[layer] * torch.mean((target_gram - style_gram)**2)
style_loss += layer_style_loss / (d * h * w)
```

The loss gets normalized by dividing by the magnitudes of each vector in this last step:

```
style_loss += layer_style_loss / (d * h * w)
```

The final loss is determined that includes both content loss and style loss with this equation shown in the code below:

```
total_loss = content_weight * content_loss + style_weight * style_loss
```

To optimize the outcome of our model, initially the target image is set to a copy of the content image using the code below:

```
target = content.clone().requires_grad_(True).to(device)
```

In effect, our target image begins as the house image through each layer, will slowly be styled according to the Van Gogh painting.

The Adam optimizer is used from this line of code:

```
optimizer = optim.Adam([target], lr=0.003)
```

The optimizer only changes the image pixels since the weights remain the same. In each iteration, target features are extracted, content loss is calculated, and next style loss is calculated and then incorporated into total loss. Then, backpropagation occurs and instead of weights being updated, the actual target image is updated within each iteration of the model. The code to perform these updates is shown on the next page:

```
optimizer.zero_grad()
```

```
total_loss.backward()
```

```
optimizer.step()
```

Every 400 steps, update images were shown and images were saved for each trial for each experiment after updating the respective hyperparameters.

3. **Experimental Setup:** To run each experiment, the target image was reset. This was important because if we do not reset the target image, then we are styling the image based on the previous experiment outcome which is not what we desire. Therefore, before each experiment, the following code was rerun to ensure independence from previous trials:

```
target = content.clone().requires_grad_(True).to(device)
```

```
optimizer = optim.Adam([target], lr=0.003)
```

Note that steps used remain consistent within each group of hyperparameters for each experiment. Some experiments were repeated with different step values to see if there was a difference in the variation achieved by changing the given parameter. The steps used were mainly 300 for faster runs or 2000 for detailed runs. However, when testing the hyperparameter of step value specifically, the number of steps used was 1000 to see what varying the step value achieved.

4. Experiments:

- a. **Style Weight Experiments:** Style weights were tested at the following values $1e4$ (low) and $1e6$ (baseline) for the following step values of 2000 and 300. A high stylization value of $1e9$ was tested for 300 steps only. Also, a low style value of $1e3$ was tested for a step value of 2000 only. At a low style weight, the image looks more like the content with slightly less stylization whereas at a high style weight there is a slightly more pronounced brush stroke effect.

This line of code was changed to test style weight:

```
style_weight = 1e6
```

The target image was also reset and the training loop was run again.

The biggest difference can be seen in combining style weight differences with step differences. Continued onto the next page are two images – one that was tested using a style weight of $1e4$ and step count of 300 while the other image was generated using a style weight of $1e6$ and a step count of 2000.

Step Count of 300 and Style Weight of $1e4$



Step Count of 2000 and Style Weight of $1e6$



Clearly, the differing style weights combined with the changing amount of steps results in a drastically different image from experiment to experiment. Increasing the number of steps as well as increasing the style weights results in an image that is clearly more styled closer to Van

Gogh's painting while still preserving the content of the initial image. With a lower style weight and a lower amount of steps, the image looks substantially less styled than its counterpart.

In comparison, if we compare two other outcomes of differing style weights while preserving the number of steps, the changes in the image are much less visible. Below we show two images with identical step values of 300 but with differing style weights of $1e4$ and $1e9$.

Step Count of 300 and Style Weight of $1e4$



Step Count of 300 and Style Weight of $1e9$



The changes within each image's style weight using a fixed amount of steps are less visible, suggesting that increasing the step value makes more of a difference in the final target image than simply adjusting the style weight alone.

A similar process is repeated for changing each image's style weights but using a fixed step value of 2000. Below are two images illustrating this:

Step Count of 2000 and Style Weight of $1e3$



Step Count of 2000 and Style Weight of $1e6$



If we examine each image very closely, we can see that the image that is styled with a lower style weight has the pixels slightly further apart than the image styled with a higher style weight. The differences are very small compared to increasing the step value in conjunction with this change.

- b. Step Count Experiments:** Step count experiments were also performed with a fixed style weight of $1e6$ in each experimental run. Step count values were tested with values of 300, 1000, and 2000 and the results for each are shown below and continued onto the next page.

Step Count of 300 and Style Weight of $1e6$



Step Count of 1000 and Style Weight of $1e6$



Step Count of 2000 and Style Weight of $1e6$



Comparing the results of the step values of 300 with 2000 yields a greater difference. This suggests that one must increase the step values substantially to notice an increase in styling. The main drawback to this approach is that increasing the step size results in a slower model run. In my experience, using a step size of 2000 would take 15 minutes at least while using a step size of 300 would take closer to 3 minutes.

To test varying step count values the number of steps calculated was changed in the for loop for the model. The code changed is shown below:

```
steps = 2000
```

and was replaced with each experimental run.

- c. **Content Layer Experiments:** To modify the code for the content layer experiments, first I ensured that the `get_features()` function extracted the specific content layer I wanted to test using the code here:

```
layers = {  
    '0': 'conv1_1',  
    '5': 'conv2_1',  
    '10': 'conv3_1',  
    '19': 'conv4_1',  
    '21': 'conv4_2',  
    '28': 'conv5_1'
```

}

Next, I changed only the content loss line to include the feature I wanted to test. This was done by replacing the content layer I wanted to test in this code:

```
content_loss = torch.mean((target_features['conv5_1'] -  
content_features['conv5_1']**2)
```

Per usual, I reset the content image prior to testing and rerunning the test loop.

The content layers that were tested were content layers of conv2_1 (shallow), conv3_1 (medium), and conv5_1 (deep). A step layer of 2000 was used throughout these experiments, and a style weight was fixed at 1e6. While using shallow layers, simpler features were more pronounced such as edges, textures, and fine details. With medium layers, shapes and patterns were detected, and with deep layers, objects and structure were more pronounced. Below and continued onto the next page were the outcomes of these experiments.

Step Count of 2000 and Content Layer of conv2_1



Step Count of 2000 and Style Weight of conv3_1



Step Count of 2000 and Style Weight of conv5_1



In examining these photos, the biggest difference can be seen between the layers conv2_1 and conv5_1. The shallow layer shows much more pronounced lines and edges while the deep layer is fully blends and incorporates the style strokes.

- 5. Conclusion:** Overall, I successfully implement a style transfer between images using the VCG19 network based on the sample Gatys paper. The process successfully extracted the style from the Van Gogh painting and applied it to the content of the photograph of the house. This was achieved using a similar technique in the paper and by minimizing both the style and content loss. A gram matrix was used to extract style representations regardless of pixel location in the image and then was applied to the content of the alternate image.

By monitoring hyperparameters such as steps, style weight, and content layers, a differing image was achieved. The biggest difference observed was when the steps were decreased to 300, less styling was applied compared to when the steps were at 2000, a much more sophisticated style was present in the target image. On the other hand, small changes in style weight alone had little effect on the resulting image while combining style weight changes with step changes resulted in the greatest difference in the target image.

These experiments have deepened my understanding about the vast applications that convolutional neural networks have and how important they are to our overall impact as engineers.

Sources:

- <https://byronsmuse.wordpress.com/wp-content/uploads/2016/07/1888-vincent-van-gogh-the-sower.jpg>
- <https://www.mightysmallhomes.com/uploads/image5.jpg>
- chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/Gatys_Image_Style_Transfer_CVPR_2016_paper.pdf
- Raschka, S., Liu, Y., Mirjalili, V., & Dzhulgakov, D. (2022). Machine learning with PyTorch and scikit-learn: Develop machine learning and deep learning models with Python. Packt Publishing.
- See ChatGPT conversation using the free version of ChatGPT in [the attached GitHub URL](#) in the sources folder